HTML – Hypertext Markup Language

Kommen wir zur „Sprache des Internets“, der Hypertext-Auszeichnungssprache HTML. Ein Beispiel:

```
<!DOCTYPE html>
<!-- Ein Kommentar -->
<html>
    <head>
        <title>Webseiten</title>
         <meta charset="utf-8">
         <meta name="referrer" content="no-referrer">
    </head>
    <body>
        <h1>HTML-Hypertext Markup Language</h1>
        <p>Kommen wir zur „Sprache des Internets“, der Hypertext-Auszeichnungssprache HTML.
              Ein Beispiel</p>
         <p>Schickes Bild:<br />
          <img src='./bilder/toll.jpg' /><br />
          <a href="toll.html">Hier klicken</a>
         </p>
    </body>
</html>
```

Starttag

Endtag

Die Sprache HTML besteht aus „Tags“: ein Tag-Name in spitzen Klammern[1]. Beachte, dass dabei zwischen der öffnenden Klammer und dem Namen kein Leerzeichen stehen darf. HTML ist ansonsten sehr entspannt im Umgang mit Leerzeichen, aber nicht an dieser Stelle. HTML ist nicht case-sensitive, Du darfst <body>, <BODY> und sogar <bODy> schreiben.

Die meisten HTML-Befehle dienen dazu, einen Text „auszuzeichnen“[2], dass heißt sie geben ihm eine Bedeutung oder zeigen an, wie er in die Struktur des Dokuments gehört. Damit klar ist, auf wie viel Text sich ein Tag bezieht, gibt es ein Starttag und ein Endtag. Das Endtag unterscheidet sich vom Starttag durch einen Slash vor dem Namen:

```
<title>Webseiten</title>
```

kennzeichnet damit „Webseiten“ als den Titel der Webseite (der in der Fensterüberschrift des Browsers angezeigt wird). Das Tag <p> ist ein „Strukturtag“, es fasst einen Text zu einem Absatz zusammen.

```
<p>Kommen wir zur… </p>
```

Es gibt auch Tags die nichts umschließen, sondern etwas einfügen. Dazu gehören zum Beispiel `<img .. />`, dass ein Bild einfügt oder der Zeilenumbruch `<br />`. Dass diese Tags nichts umschließen sondern selbstabschließend sind kann man explizit aufschreiben indem man einen Slash vor die schließende Klammer setzt (das Leerzeichen vor dem Slash ist ein Muss!). Man darf dass aber auch sein lassen, `<br>` und `<br />` sind gleichwertig.

Jedes Tag kann zusätzlich noch Attribute enthalten, bei einem Bild ist das zum Beispiel der Dateiname (Attribut `src`). Diese Attribute stehen innerhalb der spitzen Klammern des öffnenden Tags. Die Werte für diese Attribute muss man in der Regel in

1 auch bekannt als Kleiner- und Größer-Zeichen
2 einfügen → englisch „markup“ → HTML = HyperText Markup Language

CC BY-SA 3.0 DE Sascha Mertens

Anführungszeichen setzen, dabei ist es egal, ob man einfache oder doppelte Anführungszeichen nimmt (siehe Beispiel).

HTML-Dokumente eingeben

Ein HTML-Dokument kann man mit jedem beliebigen Texteditor schreiben und dann zur Kontrolle in einem Browser öffnen. Die meisten „größeren" Texteditoren (notepad++, gedit,atom) machen dabei den Code auch bunt (Syntax Highlighting). Eine Entwicklungsumgebung wie netbeans bietet außerdem Zusatzfunktionen wir Templates, Code-Ergänzung (Strg-Space !) oder Drag-and-Drop Paletten. Für den Test von HTML/CSS-Codeschnipseln bieten sich auch Online-Tools wie https://codepen.io/pen/ an, die eingegebenen Code live rendern[3] (und ebenfalls Code ergänzen):



Grundbestandteile eines HTML-Dokuments

Das Minimum für eine Seite sind der Dokumententyp, der Kopf (head) und der Körper (body). Der Dokumententyp gibt an, in welcher HTML-Version die Seite geschrieben ist,

```
<!doctype html>
```

kennzeichnet einen Seitenquelltext als HTML 5-Dokument. Im Kopf (HEAD) stehen Information **über** die Seite, also Dinge, die nicht im Hauptfenster des Browser zu sehen sind. Der Body enthält den eigentlich Inhalt (Content) der Webseite.

Als Minimalkopf empfehle ich einen Titel und die Angabe über die Kodierung der Quelltextdatei. Das Beispiel auf Seite 1 zeigt solch einen Head. Durch Angabe der Kodierung kann man Umlaute und bestimmte Sonderzeichen direkt im Quelltext

3 Wer ein ähnliches Feeling mit Netbeans haben möchte: füge im <head> die Zeile
<meta http-equiv="refresh" content="3">
ein und öffne die editierte Datei von Anfang an im Browser – sie wird dann alle 3 Sekunden neu geladen. Wenn Du jetzt noch regelmäßig Strg-S drückst, siehst Du Deine Seite fast live mitwachsen. Die Zeile musst Du aber irgendwann auch wieder entfernen….

CC BY-SA 3.0 DE Sascha Mertens

verwenden. Eine auf einem Windowsrechner erstellte Datei ist meistens iso-8859-15 kodiert, unter Linux kommt oft utf-8 zum Einsatz.

Aus Datenschutzgrund rate ich außerdem zur Verwendung von

```
<meta name="referrer" content="no-referrer">
```

Diese Angabe verhindert, dass beim Anklicken eines Links die Adresse der Seite mitgeschickt wird, auf der der Link steht. Dieses „referrer leaking“ ermöglicht nämlich ansonsten Rückschlüsse auf das Surfverhalten des Nutzers. Damit wäre ein Minimalkopf:

```
<!doctype html>
<html>
  <head>
      <meta charset='utf-8'>
      <meta name="referrer" content="no-referrer">
     <title>:: Materialseite :: Sascha Mertens ::</title>
  </head>
  <body> <!-- … -->
```

HTML zur Strukturierung – bunt wird’s später!

Ursprünglich wurde HTML entwickelt, damit Wissenschaftler im damals noch jungen Internet Texte über Ihre Arbeit austauschen konnten. Es ging also um wissenschaftliche Texte und nicht um hippe Werbeanzeigen oder Onlinespiele. Dementsprechend sind die HTML Befehle der ersten Stunde dazu gedacht, Text zu strukturieren und zu formatieren. Ursprünglich gab es daher auch Tags, um Text bunt, fett oder blinkend zu machen. Mit steigenden Ansprüchen an das Layout wurde dieser Ansatz zu unflexibel und man ist dazu übergangen, dass man mit HTML nur noch **strukturiert** und mit einer zweiten Sprache (Cascaded Stylesheets → CSS) **formatiert**. Das klingt vielleicht unnötig kompliziert, ist aber mächtiger und programmierfreundlicher. Ein Beispiel: früher schrieb man

```
<h1><color="red"><b>Intro</b></color></h1> <!-- rot und fett -->
```

und das sieht schön simpel aus. Aber man stelle sich vor, es gäbe die roten, fetten <h1> 300mal in 5 verschiedenen Dateien und der Chef/Kunde kommt und will sie ab sofort in Mintgrün. Und morgen kommt er und will sie nicht fett aber kursiv. Und übermorgen unterstrichen. Man müsste dann jedesmal mit „Suchen und Ersetzen“ in mehreren Dateien die Änderungen durchführen. Macht man dabei Fehler, sehen die Überschriften auf verschiedenen Seiten des eigenen Auftritts plötzlich unterschiedlich aus. Mit HTML + CSS steht in der HTML-Datei nur noch

```
<h1>Intro</h1>
```

und es gibt eine CSS-Datei, in der steht

```
h1 { color:red; font-weight:bold}
```

Da üblicherweise dieselbe CSS-Datei für alle HTML-Dateien eines Auftritts verwendet wird, kann man das Aussehen der ganzen Website[4] durch das Ändern in einer zentralen

4 Das ist kein Schreibfehler: Webseite = eine Webseite, Website = Internetauftritt, bestehend aus mehreren Webseiten.

CC BY-SA 3.0 DE Sascha Mertens

Datei kontrollieren. Ändert man jetzt zum Beispiel die Farbe (color:green) würde das für **alle** h1-Überschriften gelten – so wie gewünscht.

Das Prinzip ist also: mit HTML den Text strukturieren, mit CSS die Strukturelemente formatieren. Das ist übrigens derselbe Ansatz, den Textverarbeitungsprogramme verfolgen: Formatvorlagen zuweisen, um den Text zu strukturieren und dann in den Einstellungen der Formatvorlagen die Textteile gesammelt formatieren.

Für den Unterricht bedeutet das: auch wer HTML-Tags zum formatieren kennt, benutzt diese nicht. Da Ihr CSS erst später lernt müsst Ihr die Formatierung der Seite am Anfang halt so hinnehmen, wie sie in den Standardtags verankert ist.

Sonderzeichen

Viele Sonderzeichen lassen sich nicht direkt im Quelltext angeben, entweder, weil sie gar nicht über die Tastatur eingegeben werden können, oder weil sie Teil von HTML-Kommandos sind. Die Zeichen < und > werden ja zum Beispiel vom Browser als Anfang bzw. Ende eines Tags interpretiert, sie werden also grundsätzlich nicht in der Ausgabe erscheinen. Will man doch einmal ein solches Zeichen ausgeben, kann man dies auf jeden Fall dadurch erreichen, dass man den Unicode des Zeichens nachschlägt und das Zeichen dann in dem Format

```
&#[nummer];
```

angibt[5]. Die Unicode Nummer kann man auch hexadezimal angeben, dann setzt man noch ein x davor, also

```
&#x[nummer];
```

Außerdem gibt es in HTML für sehr viele Sonderzeichen Namen, deren Verwendung den Code lesbarer machen. Damit gibt es für jedes Sonderzeichen also 3 Möglichkeiten. Beispiele:

Zeichen	Beschreibung	HTML - Name	HTML Unicode	HTML Unicode hexad.
©	Copyright-Zeichen	©	©	©
±	Plusminus	±	±	±
&	Ampersand, kaufm. Und	&	&	&
<	kleiner, less than	<	<	<
	erzwungenes Leerzeichen[6]			

Die grundlegenden HTML-Tags

Ich liste die wichtigsten Tags zur Strukturierung hier einfach nur auf, ohne ausführlich zu erklären, wie sie im einzelnen wirken – das erschließt sich bei den meisten aus dem Namen oder der Kurzbeschreibung und wenn nicht: ausprobieren.

5 Folglich ist auch das & ein Sonderzeichen, dass man über diesen Umweg angeben muss.
6 „Normale“ Leerzeichen, die man mit der der Space-Taste in den Quelltext werden nicht eins zu eins in der Ausgabe angezeigt. Egal wie vieler solcher Leerzeichen man zwischen zwei Worten macht – angezeigt wird immer nur eins.

CC BY-SA 3.0 DE Sascha Mertens

<h1> , <h2>...	Überschriften	Ein HTML Crash Kurs
	Absatz	Es grünt so grün.....
	nicht vorformartierter Absatz, Kasten	Es grünt so grün.....
	nicht formatiertes Textfragment (das man dann mit CSS formatieren kann)	Diese Antwort istrichtig!
	„ordered List“ → nummerierte Aufzählung	Wir fordern - HTML für alle - Abschaffung des Compilerzwangs
	„unordered List“ → nicht nummerierte Aufzählung	
	„List Item“ Aufzählungspunkt einer Liste	
	Etwas betonen	ganz wichtig
  , 	„Break“ → Ein Zeilenumbruch[7]	Und dann BANG hat er einfach den Browser zugemacht <hr />
<hr> , <hr />	„Horizontal Rule“ → Eine horizontale Linie über die ganze Seitenbreite	

Übung

1. Kopiere Dir die Textdatei aus „vorlage1“. Mache daraus eine HTML-Seite (in eine HTML Datei kopieren oder per „Speichern unter“ mit Endung .html speichern. In der Datei sind Vorgaben vom Chef (→ von mir ;)). Suche die Vorgaben, die mit „A1“ beginnen (Suchfunktion des Editors !) und setze sie in HTML-Tags um.
   Verwende als Editor gedit oder netbeans (mit einem HTML5-Projekt).

Links und URLs

Die Verknüpfung zwischen Webseiten geschehen durch Links, diese werden mit dem Anchor-Tag <a> erstellt, wobei das href-Attribut die Linkadresse angibt (href → Hyertext Reference). Ein Anchor-Tag kann grundsätzlich alles mögliche umschließen, also sowohl ein Stück Text, als auch Text und ein Bild oder nur ein Bild oder einen ganzen Absatz.

```
<a href="http://www.html.net/">Hier ist ein Link zu HTML.net</a>
```

Als Verknüpfungsziel muss man eine gültige URL (Uniform Resource Locator) angeben. Eine **vollständige** ,absolute URL hat diese Form:

```
protokoll://hostname/verzeichnis/verzeichnis.../dateiname
```

Als Protokoll wird man in den meisten Fällen http oder https verwenden, es gibt aber auch noch andere Möglichkeiten, zum Beispiel ftp:// .

Will man auf Seiten auf dem eigenen Server verweisen, so kann man eine relative URL verwenden. Dabei lässt man Protokoll und Servernamen weg und gibt das Verzeichnis relativ an:

```
<a href="seite2.html">Ein Link zur Seite 2</a>
```

ist ein Link auf die Datei seite1.html, die im gleichen Verzeichnis liegen muss wie die Datei, in der der Link steht.

7 Zeilenumbrüche, die man mit mit der Returntaste in den Quelltext einfügt, werden nicht angezeigt.

CC BY-SA 3.0 DE Sascha Mertens

```
<a href="../index.html">Startseite</a>
```

verweist dagegen auf die Datei index.html im übergeordneten Verzeichnis, und

```
<a href="../beispiele/b1.html">Beispiel 1</a>
```

auf eine Seite im Verzeichnis `beispiel` im übergeordneten Verzeichnis. Angenommen, alle hier gezeigten Links stünden in einer Datei `seite1.html`, dann könnte die Verzeichnisstruktur so aussehen.

```
webseite
|-    index.html
|-    seiten
      |-     seite1.html
      |-     seite2.html
|-    beispiele
      |-     beispiel1.html
      |-     beispiel2.html
```

Übrigens: bei Links ohne Angaben des Protokolls ergänzt der Browser automatisch das Protokoll `file`. Würden wir unser letztes Beispiel ausschreiben, müssten wir das so tun:

```
<a href="file:../beispiele/b1.html">Beispiel 1</a>
```

Das ist auch der Grund, warum man bei Links auf externe Seiten das http bzw. https nicht weglassen kann – der Browser interpretiert Angaben ohne Protokoll automatisch als Dateinamen !

Verweise innerhalb eines Dokumentes

Man kann jedem Tag in einem HTML-Dokument eine ID geben und diese dann verlinken. Dazu gibt man die ID mit dem Hashzeichen an:

```
<h1>Inhalt</h1>
<a href="#einf">Einführung</a>
…
<h1 id="einf">Einführung<h1>
…
```

Eine ID kann man prinzipiell jedem Tag hinzufügen, also auch einem Absatz, einer Tabelle, einer Tabellenzelle und sogar einem Zeilenumbruch.

Man kann IDs auch verlinken, wenn man auf eine andere Seite springt, dazu hängt man einfach den Ausdruck `#<name der ID>` an die URL an:

```
<a href="http://www.firma.de/abteilungen.html#fertigung">
      Fertigungsabteilung der Firma </a>
```

Zusatzinformation (Tooltip)

Mit dem title-Attribut legt man noch den Text für ein den Tooltip fest, also ein Infofenster, das erscheint, wenn man mit der Maus über den Link fährt:

```
<a href="uebung.html" title="Mach diese Übungen, sie sind wichtig"> Übungen</a>
```

CC BY-SA 3.0 DE Sascha Mertens

Übung

2. Suche in der Webseite aus der 1. Aufgabe die Chef-Kommentare „A2“ und ergänze die entsprechenden Links.

   Erstelle direkt nach dem Titel ein Inhaltsverzeichnis: eine nummerierte Liste mit Links auf alle Überschriften (denk an die benötigten IDs in den <h2>-Tags)[8].

Bilder

Bilder fügt man mit dem Tag <IMG> ein, wobei das Attribut SRC den Link (!) zu der Bilddatei angibt. Außerdem kann man mit ALT einen Alternativtext angeben, der angezeigt wird, solange das Bild noch nicht geladen ist. In Browsern für optisch Herausgeforderte wird dieser Alternativtext außerdem vorgelesen.

Beachte, dass man als Bildquelle sowohl eine Datei auf dem eigenen Server als auch aus den Weiten des Internets angeben kann. Im letzteren Fall muss man aber die Möglichkeit erwägen, dass das Bild irgendwann weg ist.

```
<img src="bilder/mhmbml.jpg" alt="Mein Haus, mein Boot, mein Laptop" />
<img src="http://commons.wikimedia.org/wiki/File:Leslie_Lamport.jpg"
      alt="Leslie Lamport, der Schöpfer von LaTeX ">
```

Und nur zur Erinnerung: auch Bilder können in einem Link stehen:

```
<a href='http://www.wara.de'>  <img src="bilder/wara.png" />  </a>
```

Übung

3. Ergänze in Deiner Seite aus Aufgabe 1 die Bilder (→ Chef-Kommentar A3).

Tabellen

Tabellen entstehen durch Verschachteln der richtigen Tags:

- die Tabelle selber erstellt man als <table>
- eine Tabelle enthält einem Tabellenkopf und einem Tabellenbody (<thead> und <tbody>)
- diese bestehen aus ein oder mehreren Zeilen <tr> (für „table row“)
- die wiederum aus mehreren Zellen bestehen: <td> für „table data“ oder <th> für Table Heade (Zelle im Tabellenkopf, man kann aber auch hier <td> benutzen

Alles in allem also:

```
<table>
      <thead>
            <tr><th>x</th><th>f(x)</th></tr>
      </thead>
      <tbody>
            <tr><td>1</td><td>1</td></tr>
            <tr><td>2</td><td>4</td></tr>
            <tr><td>3</td><td>9</td></tr>
            <tr><td>...</td><td>...</td></tr>
```

8 Zum Ausprobieren einfach das ganz im Browser so sehr vergrößern, dass man selbst diesen kurzen Text scrollen muss.

CC BY-SA 3.0 DE Sascha Mertens

```
      </tbody>
</table>
```

Die Unterteilung in thead und tbody[9] kann man auch weggelassen, sie bietet aber bessere Möglichkeiten, die Tabelle mit JavaScript zu manipulieren[10].

Bitte beachtet: <table> ist dazu da um tabellarische Daten als Tabelle anzuzeigen und nicht, um das Layout der Seite zu erstellen. Das findet sich zwar als Tipp immer noch in älteren Tutorials und Foren, entspricht aber nicht mehr der modernen Herangehensweise. Lediglich beim „Layouten“ eines Formulars (kommt noch) ist es meiner Meinung nach noch legitim eine Tabelle als Layouthilfe zu missbrauchen.

Übungen

4. Schreibe eine Seite hoerprobe.html, die etwa so aussieht:



Willkommen zum Wara MP3 Shop

	Song	Kaufen
▶	The Linux Blues	€
▶	Apfelbrei & Dosenkost	€
▶	Hasta la vista, Vista	€
▶	Fenster zur Hölle	€

Die Bilder logo.png, play.png und buy.png kannst Du Dir aus „vorlage2“ kopieren. Lege sie in dasselbe Verzeichnis wie die HTML-Datei. Der Text Wara ist ein Link auf https://www.wara.de Sowohl die Play-Icons als auch der Name der Songs sollen gleichzeitig Links auf MP3 Dateien in einem Unterverzeichnis 'mp3' sein. Lege einfach mit dem Datei-Explorer (oder mit dem Konsolenbefehl touch) vier MP3-Dateien an (leere Dateien, es muss nur was zum verlinken da sein). Das ganze müsste dann so aussehen:

9 Es gibt sogar noch <tfoot>, aber Tabellenfußzeilen werden eher selten benutzt, deshalb erscheint das Tag hier nur als Fußnote.

10 Beim Ausdrucken von Tabellen die über mehrere Seiten gehen wird außerdem ein guter Browser den mit <thead> definierten Tabellenkopf am Anfang jeder neuen Druckseite wiederholen.

CC BY-SA 3.0 DE Sascha Mertens

```
|- hoerprobe.html
|- play.png
|- buy.png
|- mp3
    |- song1.mp3
    |- song2.mp3
    |- hlvv.mp3
    |- fzh.mp3
```

In manchen Tabellen will man mehrere Zellen zu einer verbinden (so wie in Calc/Excel mit „Zellen verbinden“ passieren würde). In HTML nennt man das „Zelle aufspannen“ und eine Erklärung dazu findest Du in diesem Tutorial.

5. Verändere die Vorlage „tabelleInvoice.html“ so, dass sie so aussieht:

Invoice

Item / Desc.	Qty.	@	Price
Paperclips (Box)	100	1.15	115.00
Paper (Case)	10	45.99	459.90
Wastepaper Baskets	2	17.99	35.98
Subtotal			610.88
Tax		7%	42.76
Total			653.64

6. Ergänze die Tabelle tabelleStundenplan.html. Sie soll so aussehen wie rechts. Du musst dafür auch noch Tags und Text hinzufügen. Colspan und rowspan funktionieren übrigens auch in <th>-Zellen.

Time Table					
	Mon	**Tue**	**Wed**	**Thu**	**Fri**
Hours	Science	Maths	Science	Maths	Arts
	Social	History	English	Social	Sports
	Lunch				
	Science	Maths	Science	Maths	Project
	Social	History	English	Social	

CC BY-SA 3.0 DE Sascha Mertens

Formulare

Formulare brauchen wir, um den Benutzer Eingaben machen zu lassen. Dazu stellen Formulare die üblichen Eingabeelemente grafischer Oberflächen zur Verfügung, zum Beispiel Eingabefelder, [Radio|Check|Klick]-Buttons und Auswahllisten.

```
<form action="http://web.wara.de/~mertens/formulartest.php">
    <div>
        <label for="txemail">Email</label>
        <input type="text" name="txemail" id="txemail">
    </div>
    <div>
        <label for="txmsg">Nachricht</label>
        <textarea  name="txmsg" id="txmsg"></textarea>
    </div>
</form>
```

Email

Nachricht

Ein Formular besteht zuallererst aus einem `form`-Tag. Dieses enthält den Parameter **action**, mit dem man festlegt, an welche Adresse die Benutzereingaben geschickt werden. Dabei werden die Eingabe standardmäßig in die URL codiert. Wer das nicht möchte oder längere Eingaben abholen will, gibt als METHOD die Übertragungsart POST an:

```
<form action='….' method='POST'>
```

Die benutze ich, um das Formular in Zeilen einzuteilen. Das <label> dient zum einen dazu, die Beschriftung als solche zu kennzeichnen[11] und zum Anderen wird die Beschriftung klickbar (probier es mal aus). Dazu muss das Attribut „for“ mit der „id“ des Eingabeelementes übereinstimmen.

Das Beispiel zeigt zwei typische Vertreter von Eingabeelementen:

- das selbstschließende INPUT-Tag basiert auf einem Textfeld. Durch den Parameter gibt man an, was man genau will: text , radio, checkbox, number, email...
- manche Elemente haben ein Start- und Endtag, nämlich TEXTAREA (mehrzeiliges Textfeld), SELECT (Drop-down-Liste) und BUTTON.

Danach stellt sich schon die Frage der Anordnung. Früher hat man in Formularen oft Tabellen verwendet, um die Einzelteile aneinander auszurichten, so wie in der LED-Aufgabe. Für Quick-and-Dirty Entwürfe (wie in der Prüfung ;)) reicht das. Mehr Möglichkeiten hat man, wenn man die Ausrichtung mit CSS macht. Dafür gibt es reichlich Möglichkeiten: CSS-Grid, CSS-Table, inline-Block mit fester Länge für die Labels, Flexbox. Das kommt dann im Kapitel über CSS.

Wichtig !

Ein Eingabeelement muss immer einen Namen haben. Den Namen verwendet die Empfängerseite, um die Eingaben abzuholen!

Die ID im Beispiel dient dem Zusammenspiel mit dem LABEL-Tag. Wer im LABEL das FOR weglässt, kann sich auch die ID sparen.

7. Programmiere eine Seite mit einem Formular, bei dem man einen Terminwunsch für eine Prüfung angeben kann (siehe nachfolgenden Screenshot). Für die Zeitangaben kannst Du ein <input> vom Typ time verwenden. Der ausgewählte Tag sollte als Nummer beim Server ankommen (0 für Montag, 1 für Dienstag …).

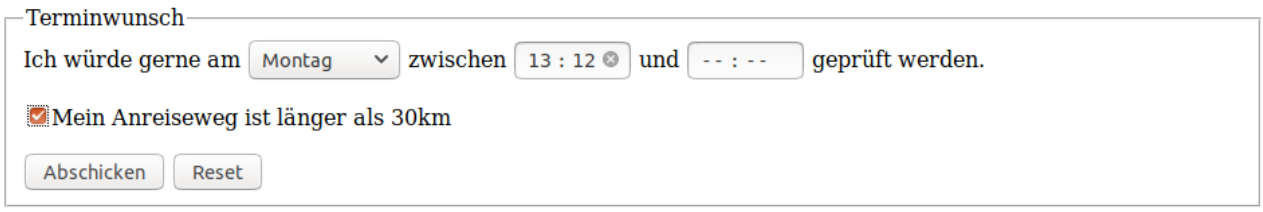
11 Wie ich zu Anfang sagte: mit HTML strukturiert man...

CC BY-SA 3.0 DE Sascha Mertens

Den Rahmen bekommst Du so hin:

```
<fieldset><legend>Terminwunsch</legend><form action=…>
…
</form></fieldset>
```

Beim Abschicken soll die Seite https://capo.wara.de/~mer/formulartest.php aufgerufen werden (das ist ein kleines Skript, das anzeigt, was Dein Formular an den Server übermittelt[12]).



8. Schreibe eine Webseite mit einem Formular wie in folgendem Screenshot. Auch dieses Formular schickt seine Daten an den Formulartester, verwendet aber POST als Übertragungsart.



12 Ausnahme: enthält Dein Formular Checkboxen, so werden diese nur angezeigt, wenn sie angeklickt sind. Für eine nicht angeklickte Checkbox wird nichts an den Server übermittelt – GAR nichts.

CC BY-SA 3.0 DE Sascha Mertens

Übersicht Formularelemente

Selbstschließende Tags			
Textfeld	`<input type='text' name='tx1' size=15>`		

Spezielle Textfelder	`<input type='number' name='tx1' min= max=6 step=1>`
	`<input type='date' name='tx1'>`
	`<input type='time' name='tx1'>`
		TT . MM . JJJJ
		-- : --	

Button	`<input type='button' value='Aufschrift' name='btn1'>`	Aufschrift	Für *type* button,submit oder reset
Checkbox	`<input type='checkbox' name='cb1' >` `<label>Toll</label>`	Toll	Text ist nicht "eingebaut" sondern wird händisch davor oder dahinter geschrieben

Radiobuttons	`<input type='radio' name='r1' value=1>`
	`<label>1</label>`
	`<input type='radio' name='r1' value=2 checked>`
	`<label>2</label>`
		1 2	Wichtig: gleicher Name, unterschiedliche Values

Umschließende Tags			
Mehrzeiliges Textfeld	`<textarea name='ta1' cols=10 rows=5></textarea>`		
Button	`<button>Klick mich</button>` `<button type='submit'>Absenden</button>`	Klick mich Absenden	Für *type* submit oder reset oder ganz ohne *type*

Drop-Down-Box	`<select name='sel1'>`
	`<option value=1>Eins</option>`
	`<option value=2 selected>Zwei</option>`
	`</select>`
		Zwei	

Auswahlliste	`<select name='sel1' size=10 multiple>`
	`<option value=1>Eins</option>`
	`<option value=2 selected>Zwei</option>`
	`</select>`
		Eins
		Zwei
			multiple erlaubt dem User eine Mehrfachauswahl

Eine ausführlichere Liste findest Du hier:

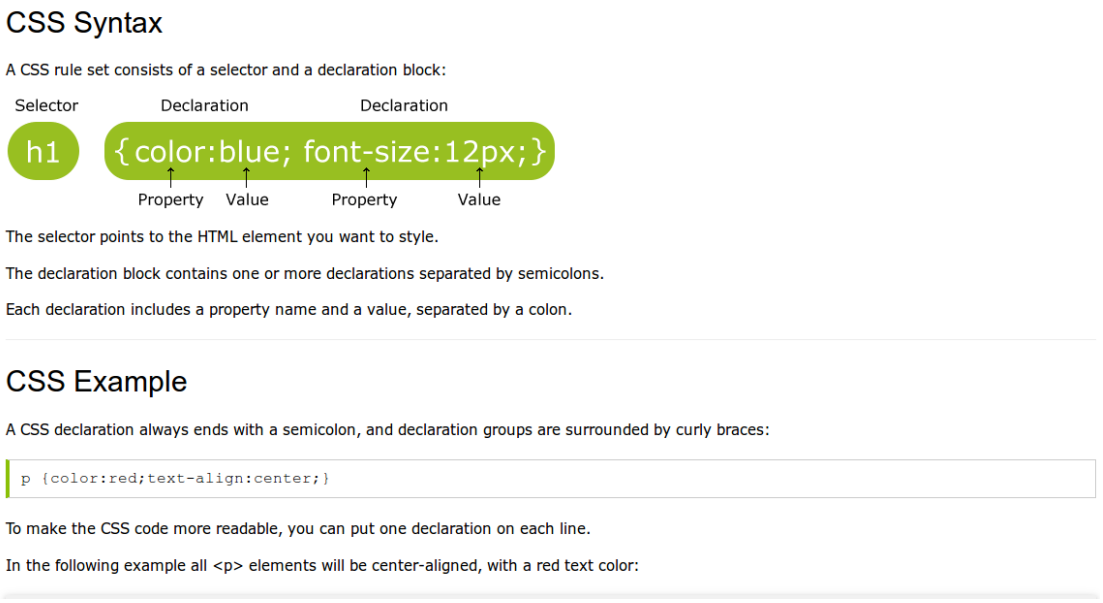
https://developer.mozilla.org/en-US/docs/Learn/HTML/Forms/The_native_form_widgets

CC BY-SA 3.0 DE Sascha Mertens

Jetzt wird's bunt – mit CSS

Übersicht CSS- Befehle: http://www.w3schools.com/css/default.asp

Gute Übungen: https://www.w3schools.com/css/exercise.asp



Die CSS-Befehle kann man übrigens auch direkt über Firefox eingeben (Wirkung wird „live" angezeigt).

- eine Datei mit Stylesheet lokal öffnen (URL beginnt mit „file://...")
- Style Editor öffnen (Tools – Web Developer – Style Editor oder Umsch-F7)
- loseditieren, gelegentlich speichern (Strg-S)

Alternativ kann man die Seite https://codepen.io verwenden (Create → New PEN) dort kann man HTML-/CSS und Javascript-Code eingeben und bekommt das Ergebnis direkt angezeigt.

CSS-Befehle in die HTML-Datei einbauen

CSS in den HTML-Head integriert (ausprobieren, prototyping)

```
<head>
      <title>Beispiel</title>
      <style type='text/css'>
            body { color : yellow; }
      </style>
</head>
```

Eine CSS-Datei verlinken (Normalfall)

```
<head>
      <title>Beispiel</title>
      <link rel="stylesheet" type='text/css' href="meinstil.css">
</head>
```

→ CSS Befehle stehen in meinstil.css

CC BY-SA 3.0 DE Sascha Mertens

Kurzübersicht der CSS-Befehle, die Du zum Überleben brauchst[13]

CSS-Befeh	für	Beispiel / Anmerkung / Typische Werte
Schrift		
font-family	Zeichensatz wählen	`font-family: Comic Sans MS,sans;` mehre Schriften als Liste angeben (Komma), als letzte eine generische Schriftart (sans-serif, serif) wer unbedingt eine Schriftart verwenden will, die es auf dem Zielrechner mglw. nicht gibt, muss einen Webfont verwenden
font-size	Zeichengröße	`font-size : 200%;` Angabe auch in pt, px etc möglich. Prozent bezieht sich auf die Standardschriftgröße, 200% → doppelte Größe
font-weight	Zeichendicke	`font-weight : 400;` `font-weight: bold;`

color	Zeichenfarbe	`color : green;`
		`color : #20a03f;`
		`color : rgb(20,160,63);`
		Farbangaben in RGB (Rot-, Grün-, Blauanteil) mit Werten zwischen 0 und 255 (00 bis FF in der hexadezimalen Darstellung)

Textausrichtung		
text-align	Horizontale Ausrichtung	`text-align : left ;`
vertical-align	Vertikale Ausrichtung	vertical-align : bottom;
Formatierung von Boxen (aka Blockelemente)		
background-color	Hintergrundfarbe	`background-color : yellow;` `background : #65432e;`

background-image
background-repeat
	Hintergrundbild**	`background-image: url(http://www.mein.de/tux2.png);`
		`background-image: url(pics/bild.jpg);`
		`background-repeat: no-repeat;`
		`background: #99bbcc url(back.jpg) repeat-y left top;`

width, height	Breite, Höhe	`div {  width : 50%;`
		`height : 3em;`
		`}`
		`span#beispiel {  width : 12ex;`
		`height : 10%;`
		`display : inline-block;`
		`}`

display	Darstellungsart	`a.wichtig { display: inline-block; }`
		`div.navi { display: inline-block; }`
		display : block → Element wird als Box angezeigt, mit Zeilenumbruch davor und dahinter, kann mit width und height formatiert werden
		display : inline → Element wird im Textfluss angezeigt, kein Zeilenumbruch, reagiert nicht auf width/height, padding und margin nur links und rechts möglich
		display : inline-block → steht inline (mit anderen Elementen in einer Zeile), kann aber in Größe (width, height) und Abstand (margin, padding) beliebig formatiert werden

13 → Pfichtwissen für die KA, Ausnahmen sind mit ** gekennzeichnet

CC BY-SA 3.0 DE Sascha Mertens

Für Angaben, die sich auf die vier Seiten der Box beziehen, gibt es immer mehrere Möglichkeiten:

für alle Seiten gleich	`margin : 1em`
für alle Seiten einzeln in einem Befehl (oben, rechts, unten,links)	`padding : 2ex 4% 3em 2em;`
(oben und unten gleich, rechts und links gleich)	`padding : 5em 10%`
für einzelne Seiten (-left, -right,-top, -bottom)	`margin-left : 10%;`

border
border-width
border-style
border-color
	Rand	`border : 2px solid #1234a0;`
		`border-style : groove;`
		`border-bottom-width : 4px;`
		`border-top-color : red;`
		`border-left : 1ex dashed green;`
		`border-style: solid dashed dotted inset;`

margin	Äußerer Abstand (zwischen Rand und Rand des Eltern- bzw. Nachbarelements)	`margin : 1em;`
		`margin-right : 20%`
		`margin : 10px 20% 4ex 10%;`

padding	Innenabstand
	Abstand zwischen Rand und Text
		`padding : 1em;`
		`padding-bottom : 0.5ex;`
		`padding : 1em 2ex;`

	Blöcke zentrieren	margin: 10% 2em; width : auto (→ linken und rechten Rand gleich machen) oder width : 40em; margin: 3em auto; (Breite vorgeben und linken/rechten Rand auto)
border-radius box-shadow	Runde Ecken und Schlagschatten für Rahmen **	border-radius : 19px; box-shadow: -0.5em -0.6em 4px #12412b; (Breite vertikal , Breite horizontal, Blur ,Farbe

Aufgaben

9. Mache Dich zunächst mit den wichtigsten Selektoren vertraut, indem Du auf https://flukeout.github.io/
   die ersten 6 Level spielst (wenn Du Zeit hast gerne auch mehr).

CC BY-SA 3.0 DE Sascha Mertens

10. Kopiere Dir die Datei cv.html und formatiere sie so, dass sie wie im Screenshot aussieht:



MAX MUSTERNERD

CURRICULUM VITAE

PERSONAL DATA

Date of birth	7.3.1983
Place of birth	Hornberg, Germany

EDUCATION

2009 - 2014	Computer Engineering Technical University of Japan in Hokkaido
2003 - 2006	Vocational Training in System Administration Walther-Rathenau-Gewerbeschule Freiburg

PROFESSIONAL EXPERIENCE

2006-2009	System Administrator Brauereigenossenschaft Hochschwarzwald
2003-2006	Apprenticeship as System Administrator Bauern IT / Bernau

Hinweise

- im HTML -Body nichts mehr Verändern
- den body auf eine Breite von 1000px und mit einem schwarzen 1px breiten Rand
- Grundschriftart Liberation Sans
- Farben: dark red, lightgray, gray, #3e4978
- Überschrift 1: Georgia, color blue, background light gray, bold, size 24px, letter spacing 1.4ex, word spacing 2em, center
- Überschrift 2: Georgia, color dark red, size 18px, letter spacing 1.4ex
- wenn Du dem label „display:inline-block“ zuweist, kannst Du mit width dessen Breite setzen.
- mit td:nth-of-type(1) {…} kann man die erste Tabellenspalte formatieren, ohne dafür eine Klasse o.ä. verwenden zu müssen

11. Die Formatierung großer CSS-Dokumente steht und fällt damit, dass man VIELE Selektoren beherrscht. Spiele Dich durch die restlichen 26 Level auf
    https://flukeout.github.io/
    damit Du in Zukunft Selektierprofi bist.
12. Formatiere die Seite uebungLayout.html. Verwende dabei alle CSS-“Pflichtbefehle“ und setze außerdem das Navigationsmenü neben den Block mit dem Inhalt.

CC BY-SA 3.0 DE Sascha Mertens

Tabellen und CSS

Bei der Formatierung von Tabellen muss man ein paar kleine Details wissen:

- die Tabelle besteht aus lauter verschachtelten Tags – man muss darauf achten was man formatiert.
  `table { border: 1pt solid black; }`
  legt einen Rahmen um die ganze Tabelle, aber keinen Rahmen um die Zellen.

  `table { padding: 0.7 em; }`

  erzeugt einen Abstand zwischen dem Tabellenrahmen und dem Inhalt der Tabelle, es legt aber nicht einen einheitlichen Innenabstand für alle Zellen fest.
- wer das Aussehen der Zellen formatieren will, muss `<td>` formatieren:

  `td { border: 1pt solid black; padding: 0.4em; }`

  legt einen Rahmen um jede einzelne Zelle und definiert Abstand zum Zelleninhalt. Allerdings liegt zwischen den Rahmen der verschiedenen Zellen noch Platz, so das kein Gitternetz entsteht (was meistens gewünscht ist). Dazu braucht es noch `border-collapse`, das wiederum auf die Tabelle angewendet werden muss:

  `table { border-collapse: collapse; }`

  Alternativ kannst Du auch mit `border-spacing:0` die beiden einzelnen Rahmenlinien zu einer zusammenfassen
- Nicht alle CSS-Eigenschaften werden für alle Tabellenelemente interpretiert. Man kann z.B, keinen Border für `tr` oder `tbody` setzen. Ein `text-align` für `tbody` setzt dagegen die Ausrichtung für alle Zellen im Tabellen-Body. Hier hilft nur – probieren.

13. Kopiere die Datei „tabellenUebungRechnung.html“ und formatiere die Tabelle. Du musst dafür die Selektoren können, die auf https://flukeout.github.io in den Levels 15 bis 18 vorkommen..

ITEM		QTY	PRICE
Don't Make Me Think by Steve Krug	In Stock	1	$30.02
A Project Guide to UX Design by Russ Unger & Carolyn Chandler	In Stock	2	$52.94 $26.47 × 2
Introducing HTML5 by Bruce Lawson & Remy Sharp	Out of Stock	1	$22.23
Bulletproof Web Design by Dan Cederholm	In Stock	1	$30.17
		Subtotal	$135.36
		Tax	$13.54
		Total	**$148.90**

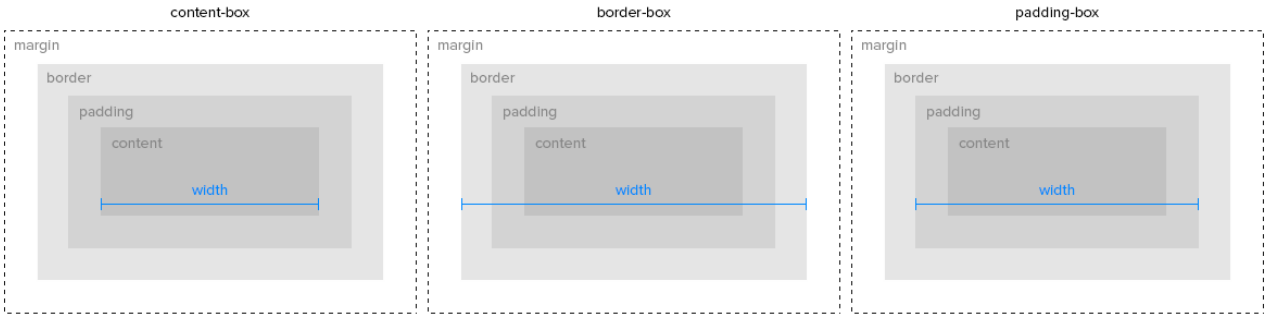
Layout Teil 1 – Displaymodes und Box-Sizing

Sobald die Webseite etwas aufwändiger wird, stellen sich Fragen wie „Wie kann ich einen Navigationsmenü neben einen Kasten mit Inhalt setzen und eine Fußzeile darunter“. Bevor man diese Fragen beantworten kann, muss man sich erstmal über die verschiedenen Layout-Arten klar werden, die man für Boxen einstellen kann. Boxen ? Ja, zuallererst muss wissen, dass (fast[14]) jedes HTML-Element eine Box definiert. Diese Box hat eine Inhalt, einen Rand, einen Innenabstand und Abstand zu den umliegenden Boxen. Aufpassen muss man dabei mit den Befehlen width und height: diese beziehen sich standardmäßig nur auf den **Inhalt** der Box. Dieses Verhalten kann man mit dem CSS-

14 Ein
 bildet zum Beispiel keine Box.

CC BY-SA 3.0 DE Sascha Mertens

Befehl box-sizing ändern, so dass sich width/height auf Inhalt + Innenabstand (padding-box) oder Inhalt+Innenabstand+Rahmen bezieht (border-box):



Wie sich eine Box in das Gesamtlayout einfügt kontrollieren wir mit dem CSS-Befehl „display“ und hier interessieren uns zunächst mal die vier Eigenschaften : none, inline, block und inline-block:

Mit display:none nehmen wir ein Element komplett von der Seite. Es nimmt dann auch keinen Platz mehr ein (was es zum Beispiel tut, wenn wir es mit visibility:hidden oder opacity:0 unsichtbar machen)

Ein „inline“-Element steht mit anderen Elementen oder Text in einer Reihe. Typische Beispiele sind , <a> oder . Inline-Elemente können einen Rahmen haben, aber man kann Ihre Größe nicht mit height oder width festlegen – sie richten sich hier ausschließlich nach Ihrem Inhalt. Einen Rand (margin) und Innenabstand (padding) kann man links und rechts setzen aber nicht für oben und unten !

*beispiel.html (/tmp) - gedit
Datei Bearbeiten Ansicht Suchen Werkzeuge Dokumente
Öffnen Speichern Rückgängig
*beispiel.html

```
<style>
 em {
        margin-left: 1em;
        border-bottom: 1px solid blue;
        font-size:140%;
        vertical-align:top;
 }
</style>

<p>
Ein Inline Element ist immer <em>genau</em> so gro&szlig; wie sein Inhalt und
"fliesst" mit dem Text (=steht mit dem Text in einer Zeile)
</p>
<p>
Unterschiedlich <em>gro&szlig;e</em> <a>Inline-Elemente</a> kann man mit vertical-align
aneinander ausrichten.
</p>
```

file:///tmp/beispiel.html
Zurück tmp/beispiel.html Duck Duck Go

Ein Inline Element ist immer *genau* so groß wie sein Inhalt und "fliesst" mit dem Text (=steht mit dem Text in einer Zeile)

Unterschiedlich *große* Inline-Elemente kann man mit vertical-align aneinander ausrichten.

Ein Block-Element ist gerade das Gegenteil. Ein Blockelement steht grundsätzlich nicht mit anderen Elementen in einer Zeile. Ein Blockelement ohne width-Angabe nimmt die volle Breite ein, unabhängig von seinem Inhalt. Man kann die Breite mit width verändern, wobei es auch bei Breiten unter 100% NICHT mit einem anderen Element in einer Reihe stehen wird. Typische Vertreter von Blockelementen sind <p>, oder <h1>.

beispiel.html

```
<style>
 p,div {
        border:2px dashed blue;
        padding:1ex;
 }

div {
        width: 30%;
        margin-top:0.2em;
}

</style>

<p>Wenig Inhalt</p><div>Etwas mehr Inhalt</div><div>Piep</div>
```

file:///tmp/beispiel.html
Zurück tmp/beispiel.html Duck Duck Go

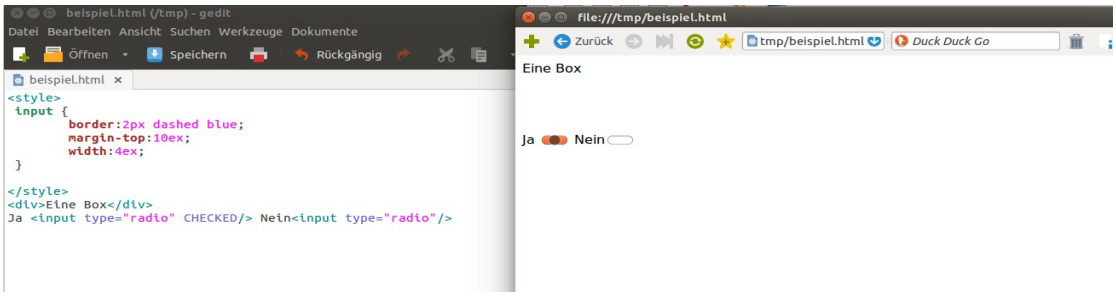
Wenig Inhalt

Etwas mehr Inhalt

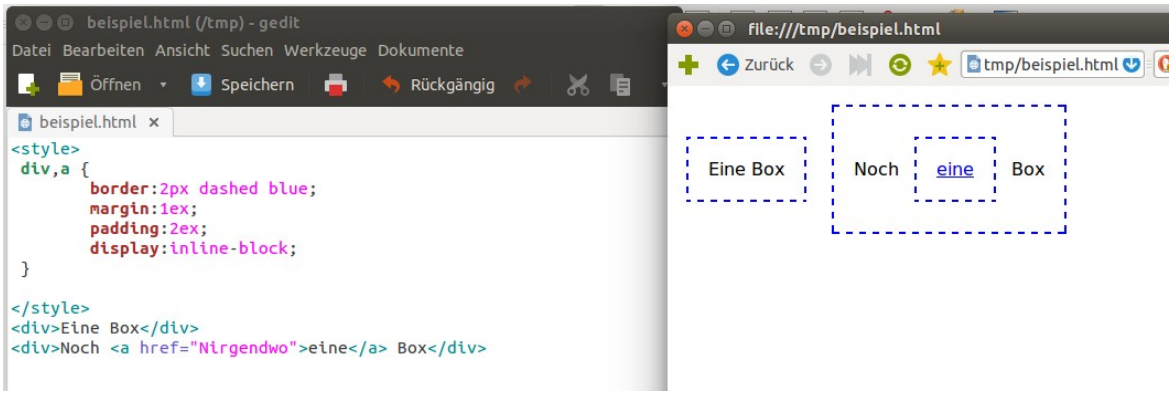
Piep

CC BY-SA 3.0 DE Sascha Mertens

Als Mittelding gibt es die Inline-Block Elemente, also Boxen, die mit anderen Boxen in einer Reihe stehen können und bei denen sich Größe, Innenabstand und Rand beliebig festlegen lassen. Die <input> Elemente sind zum Beispiel mit display:inline-block formatiert – sie stehen mit Text in einer Reihe, aber man kann sie mit width verbreitern oder Ihnen einen oberen Rand geben:



Mit der Formatierungsanweisung display:inline-block kann man aus jedem Element ein solches Multitalent machen – man kann mehrer dazu bringen, in einer Reihe zu stehen oder einen Link höher und breiter machen, als sein Inhalt vorgibt:



Positionierung der Seitenelemente & Seitenlayouts

Zum Aufbau einer kompletten Seite musst Du noch wissen, wie man die Einzelteile auf einer Seite verteilt – Kopfbereich, dann Navigationsbalken, dann Content, daneben Werbung (wenn es denn sein muss). Oder Navigation neben Content. Oder so. Dieses Thema war mal kompliziert, aber in den letzten Jahren sind reihenweise Formatierungsmöglichkeiten in CSS implementiert worden:

- „fließende" Blockelemente mit `float:[left|right]` (ähnlich Word/„seitlicher Umlauf")
- Anordnung in einem Grid (`display:grid`, wie GridPane in JavaFX)
- Ausrichtung mit Boxen (`display: flex`, wie HBox und VBox in JavaFX)

Inzwischen unterstützen zumindest alle modernen Browser das CSS-Grid, deswegen verwenden wir das auch für das Seitenlayout. Eine schöne Kurzeinführung dazu findest Du hier. Außerdem ist das hier eine gute Nachschlageseite. Eine ausführliche (englische) Anleitung mit Beispielen gibt es zum Beispiel hier.

CC BY-SA 3.0 DE Sascha Mertens

14. Mit dem CSS-Garden[15] kannst Du das Positionieren per CSS Grid üben: spiele alle Level auf https://capo.wara.de/~mer/gridgarden .
15. Kopiere Dir die Vorlage uebungLayout.zip und ergänze uebungLayout1.css so, dass die Seite so aussieht wie im folgenden Screenshot.



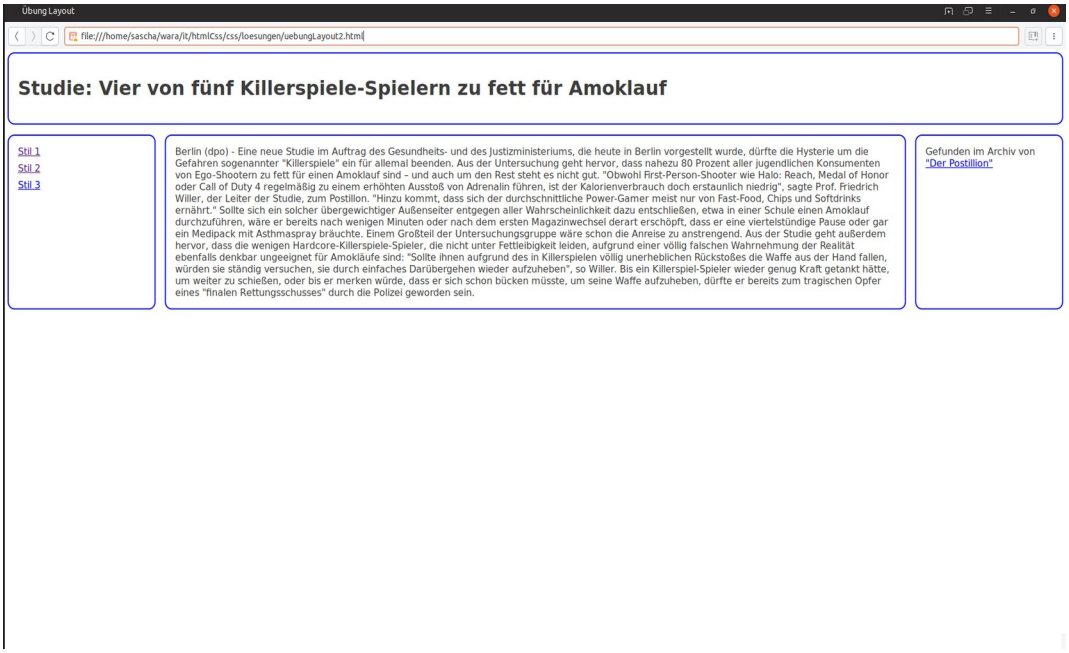
Hinweise: main hat Fenstergröße als Höhe, Navigation ca 20 Zeichen breit, Header und Footer je nach Inhalt (→ mincontent)

Übrigens: wenn Du sehen willst, wie Deine Seite auf verschiedenen Endgeräten aussieht (Tablet, Desktop, Smartphone) kannst Du sie Dir zunächst in Firefox mit verschiedenen Bildschirmgrößen anzeigen lassen: sie veröffentlichen und dann zum Beispiel diese Seite verwenden: Menü → Entwicklerwerkzeuge → Bildschirmgröße (oder Strg-Umsch-M).

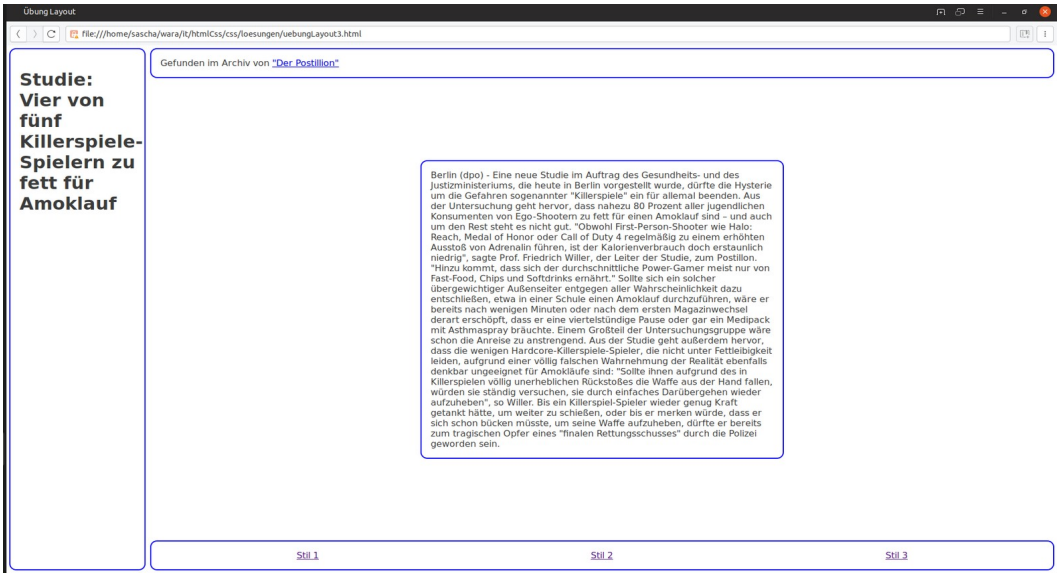
16. Passe die Dateien uebungLayout2.css und uebungLayout3.css an und bilde damit die folgenden zwei Screenshots nach.

15 Das kommt ursprünglich von https://codepip.com/games/ aber den Source-Code gibt es auf Github und ich finde es netter, nicht fremder Leute Server mit Traffic zu überziehen.

CC BY-SA 3.0 DE Sascha Mertens



Horizontale Aufteilung 1 zu 5 zu 1. Vertikal so viel Platz, wie der Text braucht



Main hat wieder Fensterhöhe, Grid mit 4 Spalten und 5 Reihen (damit um den Text Platz sein kann). Footer und Navi so hoch wie nötig, Überschrift so breit, das „Killerspiele-“ in eine Zeile passt. Text in „section“ mit ca 80 Zeichen Breite.
Hinweis: Du musst auch die Darstellung von <ul> auf display:grid setzen um die Verteilung der <li> zu kontrollieren.

CC BY-SA 3.0 DE Sascha Mertens

Formular formatieren – CSS Grid

Auch für Formulare ist das CSS-Gride ein einfacher Ansatz mit vielen Möglichkeiten.

In diesem Code fehlen die für die Reihenbildung – die Anordnung übernimmt das CSS-Grid. Dafür muss man lediglich das <form> zu einem Grid mit 2 Reihen machen. Dazu reichten die beiden Befehle



display: grid
→ zeige alles innerhalb von #f1 als Grid an

grid-template-columns: auto auto
→ zwei Spalten mit automatischer Breite

Damit ist die Anordnung erledigt, die restlichen Stilbefehle sind nur Kosmetik.

```
#f1 {
    display:grid;
    grid-template-columns:auto auto;

    max-width: 30em;

    background:rgba(218,225,100,0.5);
    padding:1em;
}

#f1 label,#f1 input,#f1 button {
    margin:1em 0;
}

fieldset {
    max-width:30em;
}
```

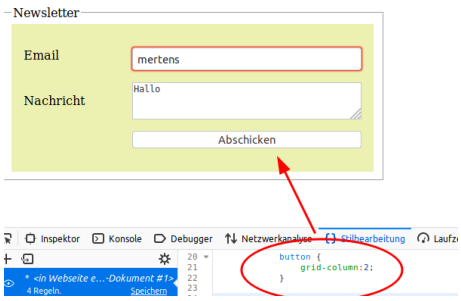
```
<fieldset>
    <legend>Newsletter</legend>
    <form id="f1" action="https://capo.wara.de/~mer/formulartest.php">

        <label for="txemail">Email</label>
        <input type="email" name="txemail" id="txemail">

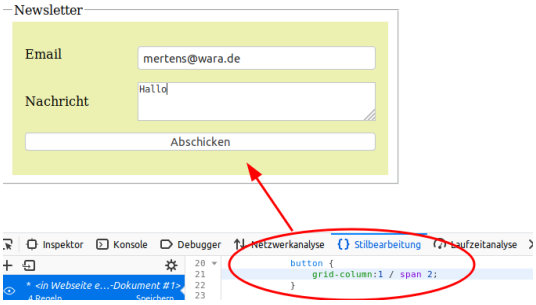
        <label for="txmsg">Nachricht</label>
        <textarea name="txmsg" id="txmsg"></textarea>

        <button type="submit">Abschicken</button>
    </form>
</fieldset>
```

Für einfache Anordnungen reicht das meistens schon. Außerdem hat man auch mit diesem simplen Code schon die Möglichkeit, per CSS zu variieren zum Beispiel indem man angibt, wohin der Button kommt:



grid-column:2 → in die zweite Spalte

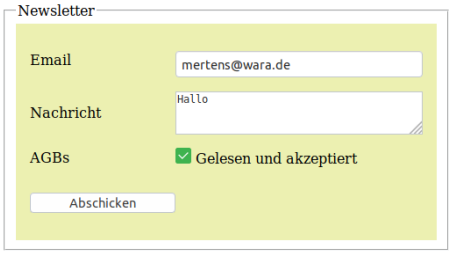


grid-column:1 /span 2
→ in die erste Spalte, aber 2 Spalten lang

Etwas komplexere Layouts bekommt man entweder durch geschicktes Verschachteln…

```
...
<label for="cbagb">AGBs</label>
<div>
    <input type="checkbox" CHECKED name="cbagb" id="cbagb">
    <label for="cbagb">Gelesen und akzeptiert</label>

</div>
<button type="submit">Abschicken</button>
...
```



… oder indem Du die Position jedes einzelnen Elemente im CSS festlegst. Beachte, dass ich dafür IDs eingefügt habe. Für die Textarea und das Textfeld (die ja die Formatierung „Spalte 2 bis 3“ teilen) hätte man auch eine Klasse vergeben können.

CC BY-SA 3.0 DE Sascha Mertens

```html
<form id="f2" action="https://capo.wara.de/~mer/formulartest.php">

    <label for="txemail">Email</label>
    <input type="email" name="txemail" id="txemail">


    <label for="txmsg">Nachricht</label>
    <textarea name="txmsg" id="txmsg"></textarea>

    <label for="cbagb">AGBs</label>

    <input type="checkbox" CHECKED name="cbagb" id="cbagb">
    <label id='lblAfterAgb' for="cbagb">
        Gelesen und akzeptiert</label>

    <button type="submit">Abschicken</button>

</form>
```

```css
#f2 {
    display:grid;
    grid-template-columns:auto 2em auto;
    max-width: 30em;
    align-items:center;
    background:rgba(218,225,100,0.5);
    padding:1em;
}

#f2 input { margin:1em 0;}

label {grid-column: 1}
#f2 input[type='email'],#f2 textarea {grid-column:2 / span 2}

#cbagb {grid-column:2}
#lblAfterAgb {grid-column:3}
```

Der Vorteil eines strikt CSS-basierten Ansatz zur Ausrichtung ist, dass Du zum Beispiel über Media-Queries das Layout anpassen kannst, wenn das Browserfenster kleiner ist/wird:

```css
@media screen and (max-width:600px){
 #f1 {
      grid-template-columns:auto
      }
}
```



bedeutet „auf ‚Bildschirmen bis 600px' gilt: Grid einspaltig"

Anregungen

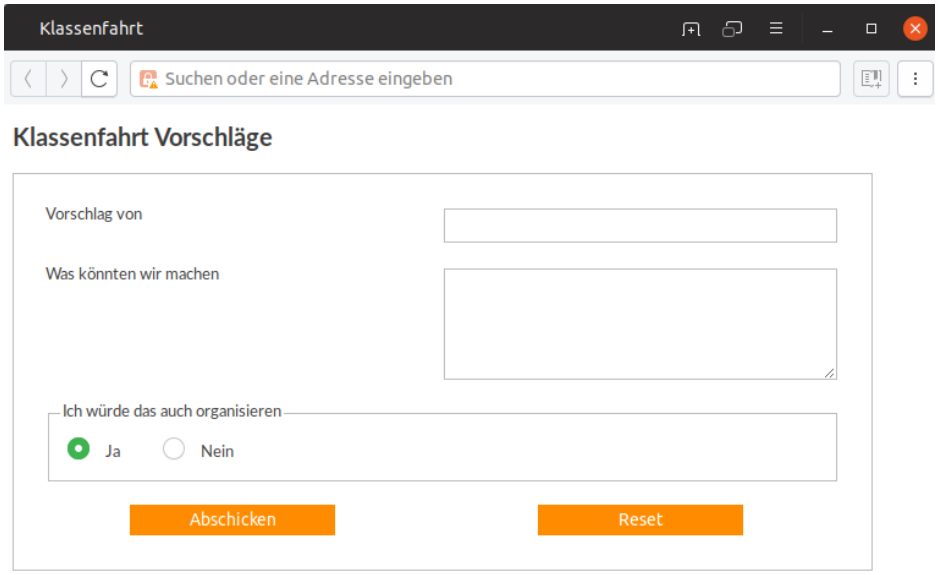
Du wirst es schon gemerkt haben – ich kann zwar CSS aber meine Ideen für gute Designs sind eher mäßig. Man findet aber reichlich Anregungen im Internet (schau einfach hin, wenn Du Dich das nächste Mal irgendwo anmeldest ;)). Hier ist auch eine Seite mit vielen netten Beispielen (und Links zu den entsprechenden HTML/CSS-Codes): https://freshdesignweb.com/css-registration-form-templates/

Übung Formulare

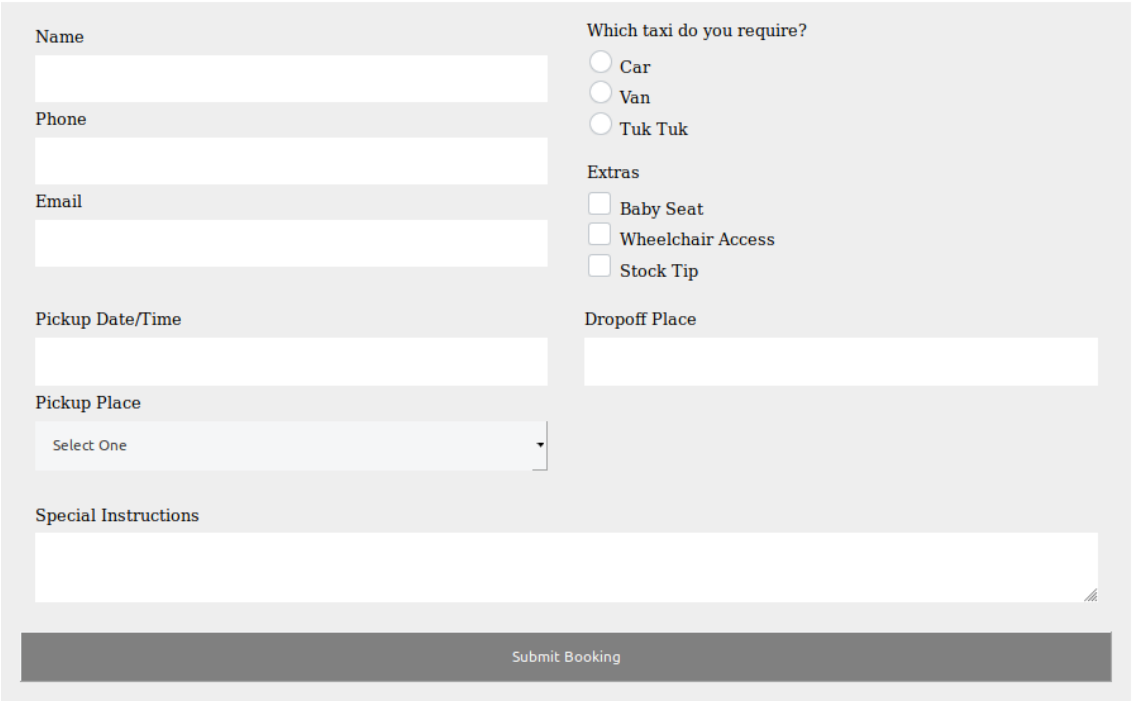
17. Formatiere das Formular „Klassenfahrt" so wie im Screenshot.

- Größe des Forms min 30em, max 50 em
- Ausrichtung mit automatischer Spaltengröße
- Farben: #bebebe und darkorange
- Schriftart: Lato
- Radiobuttons + Beschriftung mit margin auseinanderziehen
- Buttons mit justify-self zentrieren und Breite einheitlich setzen
- etwas Innenabstand für Fieldset, Texteingabefelder und Buttons wählen

CC BY-SA 3.0 DE Sascha Mertens



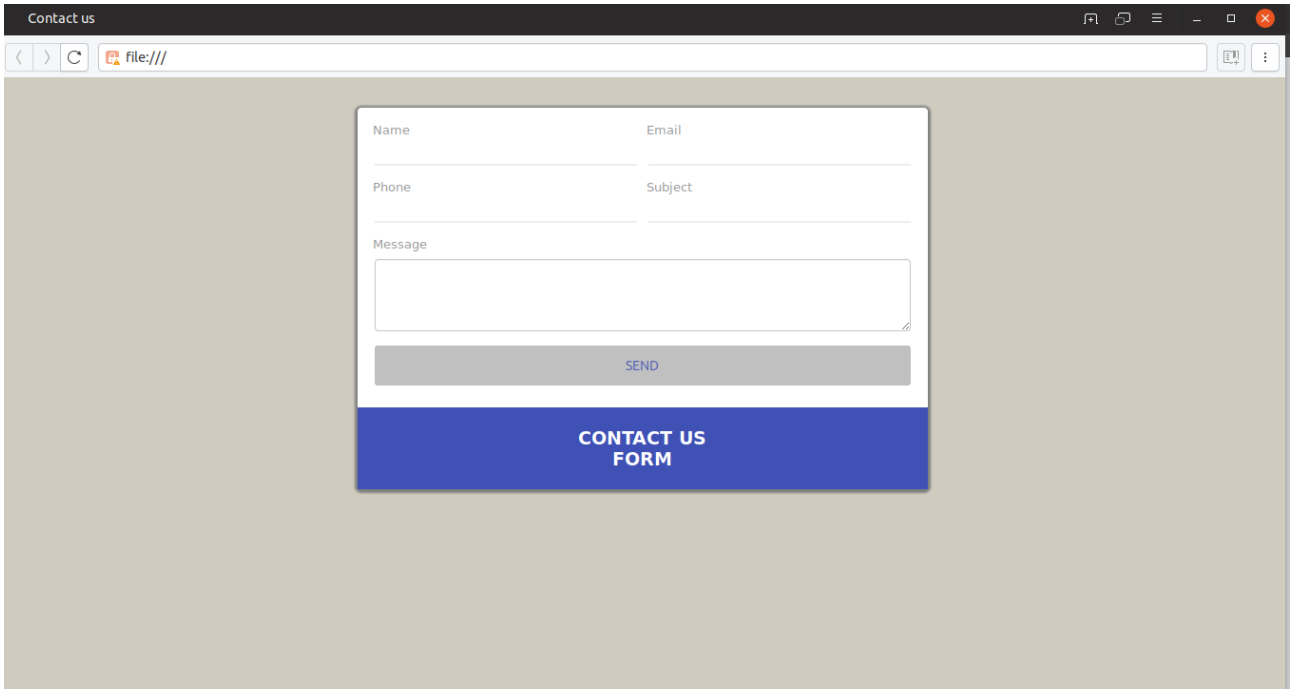
18. Lade Dir die Vorlagen taxibooking.html und taxibooking.css herunter. Dieses Formular zeigt, wie man mit Fieldsets ein Formular in mehrere Unterbereiche unterteilen kann, die man mit einem Grid anordnet. Deine Aufgabe: verteile die einzelnen Teile in dem Grid, so dass die Seite so aussieht wie im Bildschirmfoto.



Übungen für Fortgeschrittene

19. Kopiere Dir die Datei „contact1.html“. Formatiere Sie so wie im Screenshot. Verwende dabei CSS-Grid zur Anordnung der Elemente

CC BY-SA 3.0 DE Sascha Mertens

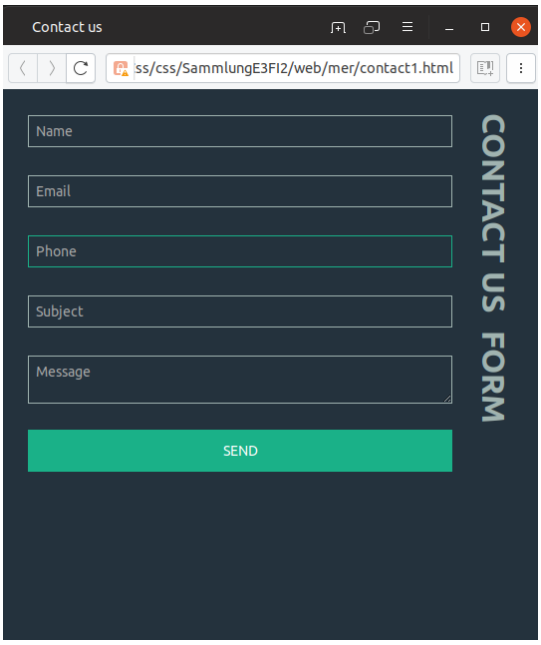


Hinweise:

- Bei den Inputfeldern nur die untere Rahmenlinie darstellen
- mit `::placeholder {opacity : 0}` wirst Du den Platzhaltertext los
- verwendete Farben: #cfcbbf,rgba(63,81,181,0.8) , #999, rgb(63,81,181)

20. Schau Dir an, was eine „media query“ ist: https://www.mediaevent.de/css/media-query.html

    Schreiben für deinen Stil aus der vorherigen Aufgabe eine Media-Query, so dass alle (1) CSS-Befehle nur gelten, wenn der Bildschirm mindestens 600px hat. Probiere es aus – wenn Du das Fenster kleiner machst, müsstest Du irgendwann die unformatierte Seite sehen.



21. Schreibe eine zweite Media-Query für Bildschirme bis 599px. Fülle diese mit CSS-Befehlen, die die Seite wie im Screenshot rechts aussehen lassen (normalerweise lässt man Seiten bei verschiedenen Größen nicht völlig unterschiedlich aussehen. Aber es geht, also machen wir es mal).
    - verwendete Farben: #a0b3b0, #1ab188 , #24323d
    - Schrift drehen mit „writing-mode“

CC BY-SA 3.0 DE Sascha Mertens

Anhang

Mehr Material / Links

Online-Editor mit Live-Darstellung

https://codepen.io

Schritt-für-Schritt-Anleitung:

https://openclassrooms.com/courses/build-your-website-with-html5-and-css3/practical-exercise-step-by-step-creation-of-a-website

Sehr ausführliches (englisches) Tutorial mit recht gutem Design

https://learn.shayhowe.com/html-css/

CC BY-SA 3.0 DE Sascha Mertens